couldn’t reach all vertices, for instance, if the traversal started at vertex v4 (or, for what is worth, v5 or v6).
In this figure, vertices are also marked with the “time” they are processed, and the time they are removed from the stack: these values are relevant for several algorithms.

To complete the discussion, listing 14.5 shows the code to perform a full dfs traversal of a graph, with restarts. It’s also possible to pass a callback to dfs, so that during traversal it can be called on each vertex visited: this can be used for a number of things, from updating the graph (changing vertices’ labels or any other attributed associated), to computing certain arbitrary operations on graphs. All you need to do, is passing the callback as an extra argument in listings 14.4 and 14.5, and call the callback as the first thing in listing 14.4.

You can also notice that, in figures 14.12-13, we also added the “times” at which vertices are visited, and exited (after that their whole adjacency list has been traversed): these values are fundamental for several applications, like computing the topological sorting (for DAGs: just order vertices by reversed out-time) finding out cycles (if a neighbor of currently-visited vertex has an in-time, but not an out-time), or computing connected components.

Listing 14.5 The dfs traversal of a graph

```
function
dfs(graph)
      #1
```